

gPCD Geometry Dynamics

May 17, 2026

1 Purpose

The current Python dynamics model is an overlap-geometry collision model. A collision is not treated as an instantaneous impulse at first contact. Instead, the current overlap area determines where the pair is along a compression or rebound path.

The old overlap-momentum response in `Dynamics.py` is no longer the active collision response. `Dynamics.py` remains as support code for contact geometry, wall geometry, lifecycle helpers, and diagnostic overlap momentum. The active model is coordinated by `Base.py` and uses `GeoDynamics.py` for the pair-level geometry calculation.

2 Rules

The model follows these project rules:

1. The model must remain an overlap-geometry model.
2. Capture files are written as `CapXXXXXX.rpt` under `C:\_DJ\gPCDData\examples`.
3. The model must remain parallel-friendly.
4. Particle velocities in the `cfg` represent incoming velocity before turnaround; they define the particles' initial momentum.
5. If a particle starts overlapped at time zero, the state is assumed to be post-turnaround and rebounding.
6. `zero_velocity_overlap_fraction` is good for designed natural collisions, but starting collisions need a calculated pair-level A_0 because we do not know how deep into collision history they already are.

3 Contact Geometry

For two particles, `Dynamics.particle_contact()` returns:

- the unit normal (n_x, n_y) from source center to target center,
- the current overlap area A ,
- the current center distance d .

For equal-radius disks, the overlap area is:

$$A(d) = 2r^2 \cos^{-1} \left(\frac{d}{2r} \right) - \frac{d}{2} \sqrt{4r^2 - d^2}.$$

Wall contacts use mirrored-circle geometry. A particle at distance s from a flat wall is treated as overlapping its mirror image at center distance $d = 2s$. The same circle-overlap area formula is then used.

4 Zero-Velocity Overlap

The zero-velocity overlap point, A_0 , is the deepest overlap reached by a collision. It is the turnaround point: compression ends and rebound begins.

For designed natural collisions, A_0 comes from configuration:

$$A_0 = f_0 A_{\max},$$

where $f_0 = \text{zero_velocity_overlap_fraction}$.

For starting overlaps, A_0 is solved from incoming momentum and overlap geometry. If the solved A_0 is shallower than the current overlap, the model promotes A_0 to the current overlap. That keeps the starting state physically possible.

5 Velocity Profile

Let A be the current overlap area and A_0 be the zero-velocity overlap area. The compression fraction is:

$$c = \text{clamp} \left(\frac{A}{A_0}, 0, 1 \right).$$

The velocity fraction is:

$$v_f = \sqrt{1 - c}.$$

This creates a parabolic collision process, analogous to the velocity profile of a ball thrown upward in a vacuum. At first contact, $c = 0$ and $v_f = 1$. At the zero-velocity point, $c = 1$ and $v_f = 0$. During rebound, the same geometry fraction maps velocity back from zero toward full rebound.

6 Relative Velocity And Momentum

For a particle pair, only relative velocity along the contact normal participates in the collision response:

$$v_n = (\vec{v}_t - \vec{v}_s) \cdot \vec{n}.$$

The reduced mass is:

$$\mu = \frac{m_s m_t}{m_s + m_t}.$$

The incoming relative normal momentum is:

$$p_n = \max(0, -\mu v_n).$$

Tangential velocity is preserved. The normal component is interpolated from the starting velocity to the zero-velocity point during compression, and from the zero-velocity point to full rebound during rebound.

7 Contact State

The Python model uses explicit contact state so a collision can remember its history across frames. This is essential because the rebound path cannot be deduced from instantaneous geometry alone. At the zero-velocity point, the current velocity may be zero; without saved first-contact information, the model would forget which way rebound should go.

7.1 Where Contact State Lives

Contact-state memory currently lives in `Base.py`:

- `Base.contact_state`: particle-particle contact state.
- `Base.wall_contact_state`: particle-wall contact state.

Both are Python dictionaries. They are reset in `Base.reset()` and are updated every simulation step.

7.2 Particle-Pair Keys

Particle-pair contact state is keyed by the sorted particle pair:

$$\text{pair_key} = (\min(i, j), \max(i, j)).$$

This means the pair has one shared Python state record, independent of which particle first reported the contact.

Each state record stores:

- `geo_phase`: compression or rebound,
- `first_contact_velocities`: starting velocity for both particles,
- `first_contact_normal`,
- `first_contact_distance`,
- `first_contact_overlap_area`,
- `geo_zero_velocity_overlap_area`,
- `geo_zero_velocity_center_distance`,
- diagnostic values such as relative normal momentum and overlap contact momentum.

7.3 Wall Keys

Wall contact state is keyed by:

$$\text{wall_key} = (\text{particle_index}, \text{wall_flag}).$$

The wall flags are:

Flag	Wall
1	left
2	right
3	bottom
4	top

Each wall state record stores:

- `geo_phase`,
- `first_contact_velocity`,
- `first_contact_normal`,
- `first_contact_distance`,
- `first_contact_overlap_area`,
- `geo_zero_velocity_overlap_area`,
- `geo_zero_velocity_center_distance`.

7.4 Why This Memory Is Needed

The geometry model needs three pieces of history:

1. the velocity at starting contact,
2. the current phase, compression or rebound,
3. the zero-velocity overlap geometry A_0 .

The current position and velocity alone are not enough. At turnaround the normal velocity is zero, so a memoryless model can lock at the zero point or misclassify the rebound as compression. Contact state prevents that by keeping the original first-contact velocity and the chosen A_0 .

8 Step Flow

The Python step flow in `Base.step()` is:

1. Begin the dynamics step.
2. Detect current particle and wall contacts.
3. Update particle-pair contact state.

4. Update wall contact state.
5. Apply geometry dynamics.
6. Move particles.
7. Detect contacts again at the new positions.
8. Update wall contact state again.
9. Clip rebound overlaps so contacts do not penetrate deeper than A_0 .
10. End the dynamics step and record the report.

9 Starting Overlaps

At reset, the model detects contacts before the first report. Starting particle-pair overlaps are initialized by `initialize_geo_starting_overlap_zero_state()`. Starting wall overlaps are initialized by `initialize_geo_starting_wall_overlap_zero_state()`.

For starting overlaps:

- the phase is set to `rebound`,
- A_0 is solved from incoming momentum,
- if needed, A_0 is promoted to the current overlap,
- the current velocity is set to the velocity that corresponds to the current overlap on the rebound path.

For starting wall overlaps, if the `cfg` velocity is already outward from the wall, the incoming first-contact velocity is inferred by reflecting that velocity across the wall normal.

10 Exit Finalization

When a contact exits, the state is removed. If the contact had reached rebound, the model finalizes the velocity to the full rebound velocity computed from the stored first-contact velocity and normal.

For particle pairs, `finalize_exited_geo_contact()` applies the final pair rebound impulse. For wall contacts, `finalize_exited_geo_wall_contact()` reflects the stored incoming wall velocity.

11 Reporting

Capture reports are compact text files. They show:

- total momentum,
- kinetic energy,
- per-particle dynamics,
- particle-particle collision diagnostics,

- wall collision diagnostics.

These reports are intended to compare the live velocity against the expected geometry-state velocity without reintroducing the old dynamics response.

12 GPU Direction

The Python contact-state dictionaries are the reference model. A GPU port must provide equivalent persistent contact memory. The simplest GPU direction is to embed source-owned contact-state slots in the particle structure:

- one slot for each `ccs` particle contact,
- later, one slot for each `bcs` wall contact.

That keeps the GPU rule parallel-friendly: each invocation writes only its own source particle state. The state must store first-contact velocity, phase, and A_0 , matching the Python dictionaries described above.